LABSHEET-9

Q1. Design a MySQL database named webapp_db with users, products, and orders tables, defining primary keys, foreign keys, and appropriate data types.

```
CREATE DATABASE webapp_db;

USE webapp_db;

CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   email VARCHAR(100) UNIQUE NOT NULL,
   password VARCHAR(255) NOT NULL,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE products (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   category VARCHAR(100),
   price DECIMAL(10,2) NOT NULL,
   image VARCHAR(255)
);

CREATE TABLE orders (
   id INT AUTO_INCREMENT PRIMARY KEY,
   user_id INT NOT NULL,
   product_id INT NOT NULL,
   quantity INT NOT NULL,
   total DECIMAL(10,2) NOT NULL,
   order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (user_id) REFERENCES users(id),
   FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Q2. Create a PHP database connection file using mysqli (or PDO) with proper error handling, and include it across multiple pages using require_once.

db.php

```
<?php
$host = "localhost";
$username = "root";
$password = "";
$database = "webapp_db";

$conn = new mysqli($host, $username, $password, $database);

if ($conn->connect_error) {
   error_log($conn->connect_error);
   die("Database connection failed.");
}

$conn->set_charset("utf8mb4");
?>
```

index.php

```
<?php
require_once 'db.php';

echo "Database Connected Successfully";
?>
```

Q3. Build a user registration page with server-side validation and store passwords securely using password_hash().

register.php

```
<?php
require_once 'db.php';

$message = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $name = trim($_POST['name']);
   $email = filter_var($_POST['email'], FILTER_VALIDATE_EMAIL);
   $password = $_POST['password'];

   if ($name == "" || !$email || strlen($password) < 8) {
      $message = "Please enter valid details.";
   } else {
      $hashedPassword = password_hash($password, PASSWORD_DEFAULT);

      $stmt = $conn->prepare(
         "INSERT INTO users (name, email, password) VALUES (?, ?, ?)"
      );

      $stmt->bind_param("sss", $name, $email, $hashedPassword);

      if ($stmt->execute()) {
         $message = "Registration Successful";
      } else {
         $message = "Registration Failed";
      }
   }
}
?>

<form method="POST">
   <input type="text" name="name" placeholder="Name" required>
   <input type="email" name="email" placeholder="Email" required>
   <input type="password" name="password"
        placeholder="Password" minlength="8" required>
   <button type="submit">Register</button>
</form>

<p><?= htmlspecialchars($message) ?></p>
```

Q4. Implement a login page that verifies credentials using password_verify() and starts a user session with session_start().

login.php

```php
<?php
session_start();
require_once 'db.php';

$message = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $email = $_POST['email'];
    $password = $_POST['password'];

    $stmt = $conn->prepare(
        "SELECT id, name, password FROM users WHERE email = ?"
    );

    $stmt->bind_param("s", $email);
    $stmt->execute();

    $result = $stmt->get_result();

    if ($user = $result->fetch_assoc()) {
        if (password_verify($password, $user['password'])) {
            session_regenerate_id(true);

            $_SESSION['user_id'] = $user['id'];
            $_SESSION['name'] = $user['name'];

            header("Location: dashboard.php");
            exit;
        }
    }

    $message = "Invalid Email or Password";
}
?>

<form method="POST">
    <input type="email" name="email" placeholder="Email" required>
    <input type="password" name="password" placeholder="Password" required>
    <button type="submit">Login</button>
</form>

<p><?= htmlspecialchars($message) ?></p>
```

Q5. Create a protected dashboard page that redirects unauthenticated visitors to the login page using session checks.

dashboard.php

```php
<?php
session_start();

if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}
?>

<h1>
    Welcome, <?= htmlspecialchars($_SESSION['name']) ?>
```

```
</h1>

<a href="products.php">View Products</a>
<a href="logout.php">Logout</a>
```

Q6. Implement a secure logout script that destroys the session, clears session variables, and removes the session cookie.

logout.php

```
<?php
session_start();

$_SESSION = [];

if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();

    setcookie(
        session_name(),
        '',
        time() - 42000,
        $params['path'],
        $params['domain'],
        $params['secure'],
        $params['httponly']
    );
}

session_destroy();

header("Location: login.php");
exit;
?>
```

Q7. Write PHP code to insert new product records into the database using prepared statements with bound parameters.

add_product.php

```
<?php
require_once 'db.php';

$message = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = trim($_POST['name']);
    $category = trim($_POST['category']);
    $price = filter_var($_POST['price'], FILTER_VALIDATE_FLOAT);

    if ($name !== "" && $price !== false && $price >= 0) {
        $stmt = $conn->prepare(
            "INSERT INTO products (name, category, price)
             VALUES (?, ?, ?)"
        );

        $stmt->bind_param("ssd", $name, $category, $price);
```

```
        if ($stmt->execute()) {
            $message = "Product Added Successfully";
        }
    } else {
        $message = "Invalid Product Details";
    }
}
?>

<form method="POST">
    <input type="text" name="name" placeholder="Product Name" required>
    <input type="text" name="category" placeholder="Category">
    <input type="number" name="price" step="0.01"
           placeholder="Price" min="0" required>
    <button type="submit">Add Product</button>
</form>

<p><?= htmlspecialchars($message) ?></p>
```

Q8. Display all product records in a styled HTML table by fetching data from MySQL with a SELECT query.

products.php

```
<?php
require_once 'db.php';

$result = $conn->query("SELECT * FROM products");
?>

<style>
table {
    width: 100%;
    border-collapse: collapse;
}

th, td {
    border: 1px solid black;
    padding: 10px;
    text-align: center;
}

th {
    background: #245c3a;
    color: white;
}
</style>

<table>
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Category</th>
        <th>Price</th>
    </tr>

    <?php while ($row = $result->fetch_assoc()) { ?>
    <tr>
```

```
        <td><?= (int)$row['id'] ?></td>
        <td><?= htmlspecialchars($row['name']) ?></td>
        <td><?= htmlspecialchars($row['category'] ?? '') ?></td>
        <td><?= htmlspecialchars($row['price']) ?></td>
    </tr>
    <?php } ?>
</table>
```

Q9. Implement an edit form that pre-fills existing record data and updates it in the database using an UPDATE query.

edit_product.php

```
<?php
require_once 'db.php';

$id = filter_input(INPUT_GET, 'id', FILTER_VALIDATE_INT);

if (!$id) {
    die("Invalid Product ID");
}

$stmt = $conn->prepare("SELECT * FROM products WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();

$product = $stmt->get_result()->fetch_assoc();

if (!$product) {
    die("Product Not Found");
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = trim($_POST['name']);
    $category = trim($_POST['category']);
    $price = filter_var($_POST['price'], FILTER_VALIDATE_FLOAT);

    if ($name !== "" && $price !== false && $price >= 0) {
        $stmt = $conn->prepare(
            "UPDATE products SET name=?, category=?, price=? WHERE id=?"
        );

        $stmt->bind_param("ssdi", $name, $category, $price, $id);

        $stmt->execute();

        header("Location: products.php");
        exit;
    }
}
?>

<form method="POST">
    <input type="text" name="name"
           value="<?= htmlspecialchars($product['name']) ?>" required>

    <input type="text" name="category"
           value="<?= htmlspecialchars($product['category'] ?? '') ?>">
```

```
    <input type="number" name="price" step="0.01"
           value="<?= htmlspecialchars($product['price']) ?>" required>

    <button type="submit">Update Product</button>
</form>
```

Q10. Implement record deletion with a JavaScript confirmation prompt and a secure PHP DELETE handler.

delete_product.php

```
<?php
require_once 'db.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT);

    if ($id) {
        $stmt = $conn->prepare("DELETE FROM products WHERE id = ?");
        $stmt->bind_param("i", $id);
        $stmt->execute();
    }
}

header("Location: products.php");
exit;
?>
```

products.php

```
<form action="delete_product.php" method="POST"
      onsubmit="return confirm('Are you sure you want to delete this product?');">

    <input type="hidden" name="id" value="<?= (int)$row['id'] ?>">

    <button type="submit">Delete</button>
</form>
```

Q11. Add pagination to the product listing page using LIMIT and OFFSET, with Previous/Next and numbered page links.

pagination.php

```
<?php
require_once 'db.php';

$limit = 5;

$page = filter_input(INPUT_GET, 'page', FILTER_VALIDATE_INT);
$page = max(1, $page ?: 1);

$totalResult = $conn->query("SELECT COUNT(*) AS total FROM products");
$total = $totalResult->fetch_assoc()['total'];

$totalPages = max(1, ceil($total / $limit));
```

```
$page = min($page, $totalPages);

$offset = ($page - 1) * $limit;

$stmt = $conn->prepare(
   "SELECT * FROM products LIMIT ? OFFSET ?"
);

$stmt->bind_param("ii", $limit, $offset);
$stmt->execute();

$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
   echo htmlspecialchars($row['name']) . "<br>";
}

if ($page > 1) {
   echo "<a href='?page=" . ($page - 1) . "'>Previous</a> ";
}

for ($i = 1; $i <= $totalPages; $i++) {
   echo "<a href='?page=$i'>$i</a> ";
}

if ($page < $totalPages) {
   echo "<a href='?page=" . ($page + 1) . "'>Next</a>";
}
?>
```

Q12. Create a search and filter feature that retrieves products by name, category, or price range using LIKE and BETWEEN clauses.

search.php

```
<?php
require_once 'db.php';

$name = $_GET['name'] ?? '';
$category = $_GET['category'] ?? '';
$min = filter_var($_GET['min'] ?? 0, FILTER_VALIDATE_FLOAT);
$max = filter_var($_GET['max'] ?? 100000, FILTER_VALIDATE_FLOAT);

$min = $min !== false ? $min : 0;
$max = $max !== false ? $max : 100000;

$stmt = $conn->prepare(
   "SELECT * FROM products
    WHERE name LIKE ?
    AND category LIKE ?
    AND price BETWEEN ? AND ?"
);

$name = "%" . $name . "%";
$category = "%" . $category . "%";

$stmt->bind_param("ssdd", $name, $category, $min, $max);

$stmt->execute();
```

```
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
   echo htmlspecialchars($row['name']) . " - ";
   echo htmlspecialchars($row['price']) . "<br>";
}
?>

<form method="GET">
   <input type="text" name="name" placeholder="Product Name">
   <input type="text" name="category" placeholder="Category">
   <input type="number" name="min" placeholder="Minimum Price">
   <input type="number" name="max" placeholder="Maximum Price">
   <button type="submit">Search</button>
</form>
```

Q13. Implement column sorting (ascending/descending) on the product table by clicking the table headers.

sort.php

```
<?php
require_once 'db.php';

$allowedColumns = ['name', 'category', 'price'];

$sort = $_GET['sort'] ?? 'name';
$order = $_GET['order'] ?? 'ASC';

if (!in_array($sort, $allowedColumns, true)) {
   $sort = 'name';
}

$order = strtoupper($order) === 'DESC' ? 'DESC' : 'ASC';

$sql = "SELECT * FROM products ORDER BY $sort $order";

$result = $conn->query($sql);
?>

<table border="1">
   <tr>
      <th><a href="?sort=name&order=ASC">Name ↑</a></th>
      <th><a href="?sort=name&order=DESC">Name ↓</a></th>
      <th><a href="?sort=price&order=ASC">Price ↑</a></th>
      <th><a href="?sort=price&order=DESC">Price ↓</a></th>
   </tr>

   <?php while ($row = $result->fetch_assoc()) { ?>
   <tr>
      <td><?= htmlspecialchars($row['name']) ?></td>
      <td><?= htmlspecialchars($row['name']) ?></td>
      <td><?= htmlspecialchars($row['price']) ?></td>
      <td><?= htmlspecialchars($row['price']) ?></td>
   </tr>
   <?php } ?>
</table>
```

Q14. Build an image upload feature for product photos with validation of file type, size, and unique file naming in PHP.

upload.php

```
<?php
require_once 'db.php';

$message = "";

if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $name = trim($_POST['name']);
   $category = trim($_POST['category']);
   $price = filter_var($_POST['price'], FILTER_VALIDATE_FLOAT);

   if (isset($_FILES['image']) &&
      $_FILES['image']['error'] === UPLOAD_ERR_OK &&
      $_FILES['image']['size'] <= 2 * 1024 * 1024 &&
      $price !== false && $price >= 0) {

      $finfo = new finfo(FILEINFO_MIME_TYPE);
      $mime = $finfo->file($_FILES['image']['tmp_name']);

      $allowed = [
         'image/jpeg' => 'jpg',
         'image/png' => 'png'
      ];

      if (isset($allowed[$mime])) {
         if (!is_dir('uploads')) {
            mkdir('uploads', 0755, true);
         }

         $filename = bin2hex(random_bytes(16)) . '.' . $allowed[$mime];
         $path = 'uploads/' . $filename;

         if (move_uploaded_file($_FILES['image']['tmp_name'], $path)) {
            $stmt = $conn->prepare(
               "INSERT INTO products (name, category, price, image)
                VALUES (?, ?, ?, ?)"
            );

            $stmt->bind_param("ssds", $name, $category, $price, $filename);

            $stmt->execute();

            $message = "Product Uploaded Successfully";
         }
      } else {
         $message = "Invalid Image Type";
      }
   } else {
      $message = "Invalid File or File Too Large";
   }
}
?>

<form method="POST" enctype="multipart/form-data">
   <input type="text" name="name" placeholder="Product Name" required>
```

```
    <input type="text" name="category" placeholder="Category">
    <input type="number" name="price" step="0.01" min="0" required>
    <input type="file" name="image" accept="image/jpeg,image/png" required>
    <button type="submit">Upload Product</button>
</form>

<p><?= htmlspecialchars($message) ?></p>
```

Q15. Demonstrate protection against SQL injection by comparing a vulnerable query with a prepared-statement version.

Vulnerable Query

```
<?php
$email = $_POST['email'];

$sql = "SELECT * FROM users WHERE email = '$email'";

$result = $conn->query($sql);
?>
```

Prepared Statement

```
<?php
$email = $_POST['email'];

$stmt = $conn->prepare(
    "SELECT * FROM users WHERE email = ?"
);

$stmt->bind_param("s", $email);

$stmt->execute();

$result = $stmt->get_result();
?>
```

Q16. Prevent Cross-Site Scripting (XSS) by escaping all user-generated output using htmlspecialchars().

index.php

```
<?php
$name = $_POST['name'] ?? '';

$safeName = htmlspecialchars(
    $name,
    ENT_QUOTES | ENT_SUBSTITUTE,
    'UTF-8'
);
?>

<form method="POST">
    <input type="text" name="name" placeholder="Enter Name">
    <button type="submit">Submit</button>
</form>
```

```
<h2>Welcome, <?= $safeName ?></h2>
```

Q17. Use cookies to implement a "Remember Me" option on the login form and display a personalized welcome message.

login.php

```
<?php
session_start();
require_once 'db.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {
   $email = $_POST['email'];
   $password = $_POST['password'];

   $stmt = $conn->prepare(
      "SELECT id, name, password FROM users WHERE email = ?"
   );

   $stmt->bind_param("s", $email);
   $stmt->execute();

   $user = $stmt->get_result()->fetch_assoc();

   if ($user && password_verify($password, $user['password'])) {
      session_regenerate_id(true);

      $_SESSION['user_id'] = $user['id'];
      $_SESSION['name'] = $user['name'];

      if (isset($_POST['remember'])) {
         setcookie(
            'remember_name',
            $user['name'],
            [
               'expires' => time() + 86400 * 30,
               'path' => '/',
               'secure' => !empty($_SERVER['HTTPS']),
               'httponly' => true,
               'samesite' => 'Lax'
            ]
         );
      }

      header("Location: dashboard.php");
      exit;
   }

   echo "Invalid Login";
}
?>

<form method="POST">
   <input type="email" name="email" placeholder="Email" required>
   <input type="password" name="password" placeholder="Password" required>

   <label>
      <input type="checkbox" name="remember">
```

```
    Remember Me
  </label>

  <button type="submit">Login</button>
</form>
```

dashboard.php

```
<?php
session_start();

$name = $_SESSION['name']
  ?? $_COOKIE['remember_name']
  ?? 'Guest';
?>

<h1>Welcome, <?= htmlspecialchars($name) ?></h1>
```

Q18. Implement a shopping cart using PHP sessions that supports adding, updating quantities, and removing items.

cart.php

```
<?php
session_start();
require_once 'db.php';

if (!isset($_SESSION['cart'])) {
  $_SESSION['cart'] = [];
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $id = filter_input(INPUT_POST, 'id', FILTER_VALIDATE_INT);
  $action = $_POST['action'] ?? '';

  if ($id && $action === 'add') {
    $stmt = $conn->prepare(
      "SELECT id, name, price FROM products WHERE id = ?"
    );

    $stmt->bind_param("i", $id);
    $stmt->execute();

    $product = $stmt->get_result()->fetch_assoc();

    if ($product) {
      $_SESSION['cart'][$id] = ($_SESSION['cart'][$id] ?? 0) + 1;
    }
  }

  if ($id && $action === 'update') {
    $quantity = filter_input(INPUT_POST, 'quantity', FILTER_VALIDATE_INT);

    if ($quantity !== false && $quantity > 0 && $quantity <= 99) {
      $_SESSION['cart'][$id] = $quantity;
    }
  }
```

```
    if ($id && $action === 'remove') {
        unset($_SESSION['cart'][$id]);
    }
}

foreach ($_SESSION['cart'] as $id => $quantity) {
    echo "Product ID: " . (int)$id;
    echo " Quantity: " . (int)$quantity . "<br>";
}
?>
```

Q19. Create a checkout script that saves the cart contents into the orders table using a database transaction (BEGIN, COMMIT, ROLLBACK).

checkout.php

```
<?php
session_start();
require_once 'db.php';

if (!isset($_SESSION['user_id']) || empty($_SESSION['cart'])) {
    die("Please login and add products to cart.");
}

$userId = $_SESSION['user_id'];

try {
    $conn->begin_transaction();

    foreach ($_SESSION['cart'] as $productId => $quantity) {
        $stmt = $conn->prepare(
            "SELECT price FROM products WHERE id = ? FOR UPDATE"
        );

        $stmt->bind_param("i", $productId);
        $stmt->execute();

        $product = $stmt->get_result()->fetch_assoc();

        if (!$product) {
            throw new Exception("Product not found");
        }

        $total = $product['price'] * $quantity;

        $insert = $conn->prepare(
            "INSERT INTO orders (user_id, product_id, quantity, total)
             VALUES (?, ?, ?, ?)"
        );

        $insert->bind_param(
            "iiid",
            $userId,
            $productId,
            $quantity,
            $total
        );

        $insert->execute();
```

```
    }

    $conn->commit();

    $_SESSION['cart'] = [];

    echo "Order Placed Successfully";

} catch (Throwable $e) {
    $conn->rollback();

    error_log($e->getMessage());

    echo "Checkout Failed";
}
?>
```

Q20. Use JavaScript fetch() (AJAX) to perform live search suggestions without reloading the page.

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Live Product Search</title>
</head>
<body>

<h2>Search Products</h2>

<input type="text" id="search"
       placeholder="Search Product">

<div id="results"></div>

<script>
document.getElementById("search").addEventListener("input", function() {

    let query = this.value;

    fetch("live_search.php?q=" + encodeURIComponent(query))
        .then(response => response.text())
        .then(data => {
            document.getElementById("results").innerHTML = data;
        });
});
</script>

</body>
</html>
```

live_search.php

```
<?php
require_once 'db.php';

$query = $_GET['q'] ?? '';
```

```
$stmt = $conn->prepare(
   "SELECT name, price FROM products WHERE name LIKE ? LIMIT 10"
);

$search = "%" . $query . "%";

$stmt->bind_param("s", $search);

$stmt->execute();

$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
   echo "<p>";
   echo htmlspecialchars($row['name']);
   echo " - ";
   echo htmlspecialchars($row['price']);
   echo "</p>";
}
?>
```